AGENT MEMORY SYSTEMS · 第三部分 · 第13讲

记忆编码、分段、摘要与压缩

Encoding · Chunking · Summarization · Compression

> 信息变短，不等于记忆更准确。
> 保留关键词，不等于保留原意。
> 重要的，是未来还能还原事实与决策。

零基础理论 | 17张原理图 | 科研助手贯穿案例 | 12道思考题与解答

《智能体记忆系统：从零基础到科研理解》
32讲系列 · 第13/32讲 · 2026年10月18日 · 版本 v1.0

《智能体记忆系统：从零基础到科研理解》 · 理论课第13/32讲 · 第三部分：记忆分类、形成与编码机制 · v1.0 · 2026-10-18

> 本讲只讲理论，不安排实验、安装或编程。所有项目人物对话、数值与评价结果都是课堂构造，用于推演原理，不代表实际科研项目、真实授权或模型实测。论文内容依文末可核查来源概述，教学示意图为原创简化模型。

学习导航：把“记住”拆成四个不同的动作

第11讲研究“信息怎样从对话、工具、文献中成为候选记忆”，第12讲研究“哪条候选值得保存”。现在来到下一个关口：**决定保存一条候选以后，到底怎样把它变成以后容易查找、容易理解、不会误用的记忆？**

这一步往往包含四个不同动作：**编码（encoding）**决定内容和元数据如何表示；**分段（chunking）**决定一大段原文怎样划分；**摘要（summarization）**决定如何提取较短的叙述；**压缩（compression）**决定在预算约束下舍弃哪些内容。四者可以同时使用，但绝不是同义词。

本讲最终要建立一条可以独立推演的因果链：原始记录 → 语义单元 → 分段组织 → 结构化编码 → 多层摘要／压缩 → 带来源的索引 → 检索与行动 → 反馈修正。每一条箭头都可能引入可追踪的损失。

FIG 01 从候选记忆到可利用记忆

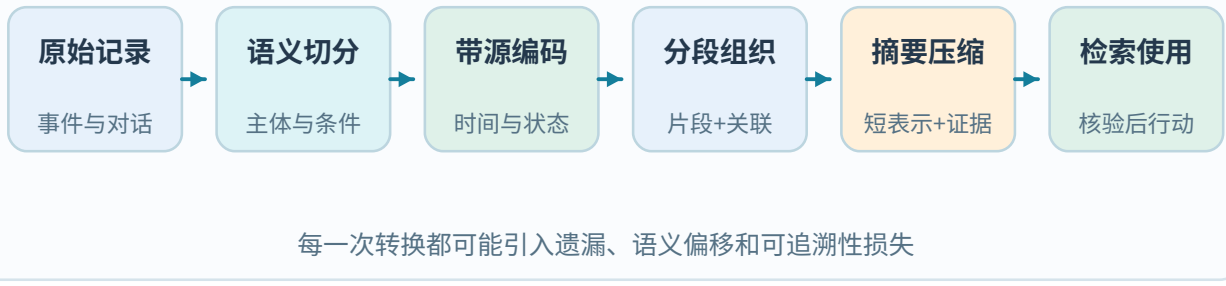


图1 记忆从原始事件到检索利用的六步转换。

贯穿案例。 假设一个科研助手连续30天协助一个“Agent Memory Systems”项目。第3天智能体提出“以后可以考虑扩样”，第8天工具显示“12次请求中有2次超时”，第12天研究者明确说“本轮暂停新增收费调用”，第17天新报告解释先前超时可能来自网络，尚未认定为模型失败。第30天研究者询问：“可以启动下一轮付费实验了吗？”下面所有编码和压缩案例都围绕这个虚构时间线。

01 编码究竟是什么意思？不只是把文字转换成向量

“编码”在计算机领域有多个层次：用UTF-8表示汉字是字符编码；把文字变成Token ID是分词编码；把文本变成向量是语义表示；把对话转成可查询的事件与主张，是**记忆的语义编码**。它们之间存在联系，却不能相互替代。

一个最简单的原文是：

> 研究者（第12天）：“本轮暂停新增收费 API 调用。旧的扩样建议留档，但不能视为批准。”

如果只生成一条Embedding向量，未来可以找到“有关实验预算”的相似内容，但单个向量本身通常不能可靠还原“谁说的、哪一天、暂停的范围、否定词、哪个旧建议被覆盖”。因此，一个面向持续科研工作的记忆条目，至少需要**人类可读的主张内容 + 来源证据 + 时间范围 + 当前状态 + 便于检索的索引**。

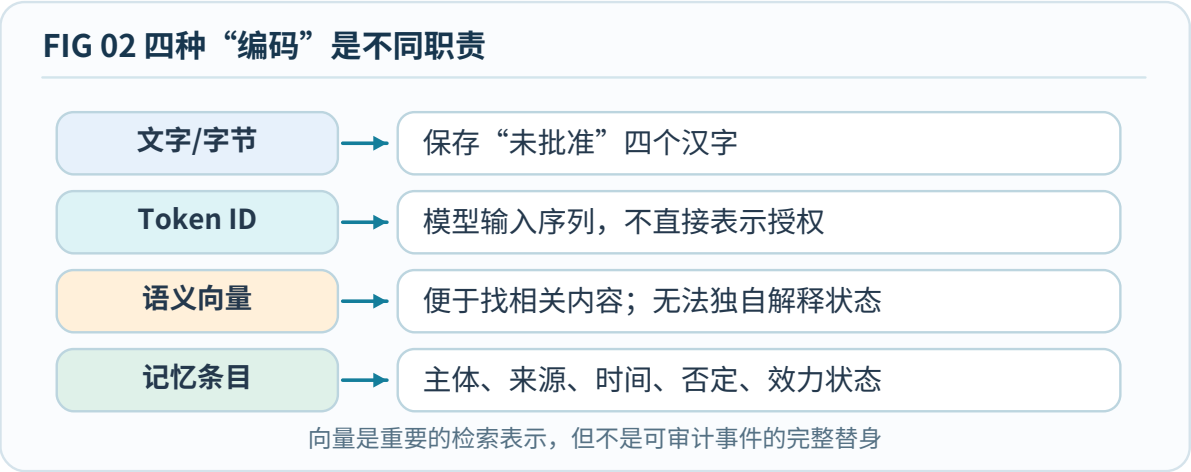


图2 四种“编码”的职责并不相同。

1.1 从句子提取语义单位

可以把原句拆成两条相互关联、但用途不同的断言：

- 断言A：研究者在第12天宣布，本轮不新增收费API调用。类型为“当前项目决策”，范围为“本轮”；
- 断言B：第3天的扩样建议可作为历史提案保存，但没有获得执行批准。类型为“历史提案状态说明”。

如果抽取器把两条断言合成“研究者批准了扩样，但暂停收费”，语义已经发生破坏。这里的关键并非文字缩短了多少，而是**语义角色、否定极性和适用条件是否正确**。

1.2 元数据像记忆的“身份证”

一条适合存储的候选卡片通常记录：`memory_id`（唯一身份）、`project_id`（项目空间）、`subject`（主体）、`claim`（断言）、`speaker`（谁提出）、`source_event_id`（原始事件定位）、`event_time`（事情发生时间）、`valid_from/valid_to`（效力时间）、`status`（有效、待核、已替代等）、`scope`（适用范围）、`confidence/evidence`（证据状态）。

注意：“第12天出现这句话”与“这条约束从第12天起对当前实验生效”属于两个不同的时间概念；当信息在第20天才被系统读取时，还要区分“获知时间”。并非所有应用都需要这些字段，但只要系统涉及跨日更新，就不能忽略时间与来源。

1.3 结构化不是把语言伪装成事实

某条候选可以写成字段齐全的数据库记录，也可以写成JSON或自然语言备忘录。字段齐全只说明**表示格式清晰**，不能使其自动成为真实结论。若来源是模型自行推断，`claim_type=INFERENCE`、`status=PENDING`就不能因格式整洁被改成`CONFIRMED`。

02　为什么要分段？先从“整本书一次搜索”想起

想象一本500页的项目日志，研究者只问“第12天对预算作了什么决定”。如果整个日志只对应一个向量，粗粒度语义可能混合了网络故障、论文讨论、预算限制和待办事项；命中后还需把大段内容送进模型。分段的直接目的，是让**检索粒度与问题所需信息的粒度相匹配**。这不是简单地让数据变短，更是对可定位性和语义边界进行设计。

2.1 三种最直观的分段方法

固定长度分段。 每500 Token切一段，不关心句子或时间边界。优点是实施和成本预测容易，缺点是可能把“不批准”中的否定词和被修饰对象切开。

对话／事件分段。 根据说话者、消息轮次、工具调用、时间点或任务节点切分。适合保留“谁说了什么、谁执行了什么”的关系，代价是各段长度不均匀。

语义／结构分段。 根据主题、论文小节、结论与证据等结构切分。适合长文档与项目总结，但主题识别可能出错，过长的单个事件仍需进一步拆分。



图3 固定、事件与语义分段的比较。

2.2 一次可以手算的固定分段

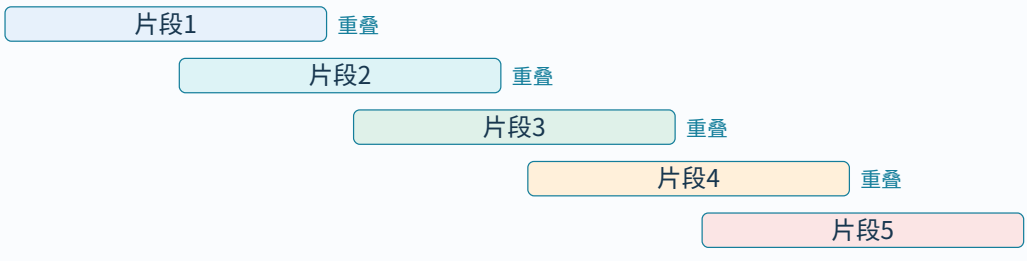
假设原文长度 `L=2,100 Token`，窗口长度 `C=500 Token`，相邻窗口重叠 `O=100 Token`。每次向前移动的**步长**为 `S=C-O=400 Token`。按从第0个位置起算，可得到：

- 第1段：0—500；第2段：400—900；第3段：800—1,300；
- 第4段：1,200—1,700；第5段：1,600—2,100。

总共是5段；索引文本总长度为 `5×500=2,500 Token`，相较原文多出 `400 Token`，也就是原文的 `400÷2,100≈19.0%`。这些增加的内容来自重叠，而不是新知识。对于本例整齐对齐的边界，可用 `N=1+ceil((L-C)/(C-O))=5` 计算。若最后一段短于500，分段总存储长度就不能简单使用 `N×500`。

重叠能降低某些边界断裂风险，但不是万灵药：增加冗余、索引规模和重复检索；也无法自动让模型知道“第3天是提议，第12天是否定当前执行”。

FIG 04 固定长度与重叠窗口的直观关系



5段总索引长度2,500 Token；重复400 Token，约为原文19.0%

图4 重叠窗口的存储冗余与边界覆盖。

2.3 两个容易犯的分段错误

第一，把工具请求与返回结果分开：一段仅存“调用命令成功”，另一段是“工具其实报超时”，后续检索只找到第一段便会误判任务成功。第二，把旧结论和新修订拆成两条互不关联的片段：语义相似度可能把旧提案当作最新有效事实。因此**分段不仅要保存局部信息，还要保留跨段关系**，例如`parent_id`、`previous_event_id`、`supersedes_id`和原始文件的字符位置。

03　分段的粒度为什么没有唯一最优值?

把分段做得很小，定位可能很精确，但关键条件可能散落在别处，导致“断章取义”；做得很大，局部证据完整，但检索结果混杂、消耗更多上下文预算。适合的粒度取决于未来问题的粒度、文档结构、检索器、模型上下文预算与可接受成本。

FIG 05 分段粒度的三种典型取舍



示意规律而非实测曲线；不存在脱离任务的固定最优长度

图5 分段粒度是定位、完整性和成本的折中。

例如研究者问“第12天暂停了哪种调用？”一个包含日期、主体和完整否定句的较小片段足以回答。若问“为什么从第8天的超时事件最终得到第17天的故障归因？”，系统可能需要一段跨事件的时间线及多个原文指针，仅靠单句片段就不够。**面向事实的问题与面向事件链的问题，对最佳粒度的要求不同。**

3.1 父子分段与多粒度索引

一种常见工程思想是保留三层：原始文档（可追溯证据）；章节或事件组（提供上下文）；细粒度片段（便于快速定位）。检索器可先找到小片段，再按`parent_id`取得父段，避免把孤立句子直接交给模型。

这并不意味着必须部署三个数据库。可以在一张表中用父子关系标注，也可以用文档存储与向量索引组合。RAPTOR（Sarthi等，ICLR 2024）提供了一种更进一步的思路：对文本片段递归聚类、摘要，构建多层树，并在推理时从不同抽象层次检索；但它研究的是特定检索增强场景，不能直接推论“树状摘要是所有Agent长期记忆的最佳存储方式”。[7]

FIG 06 多粒度摘要 + 原始片段的层次结构

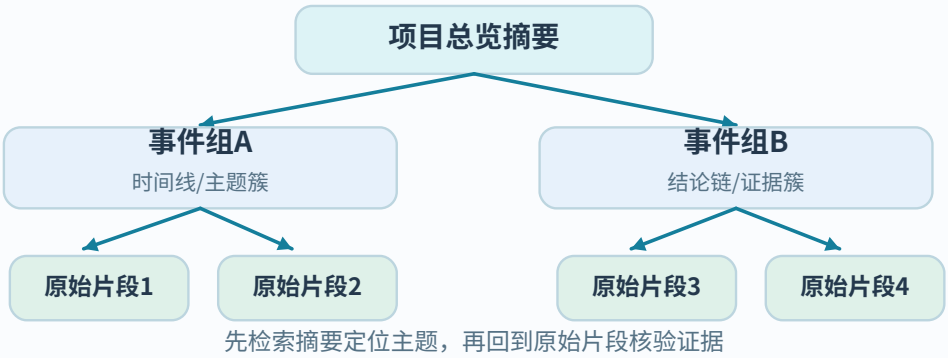


图6 多层摘要与原始证据的对应关系。

04 摘要是什么？“抽出句子”和“重写意思”并不一样

摘要的目标通常是用较短文本表达输入的关键信息，但“关键”必须相对任务定义。**抽取式摘要（extractive）**选择原文中的片段或句子；**生成式摘要（abstractive）**可以重组语言、形成更简洁的叙述。二者各有优势：抽取式更容易对齐原文，但可能保留大量冗余；生成式更流畅、能跨片段整合，却可能错误补全或改写事实。

以虚构第12天的原文为例：

原文：`研究者说：“本轮暂停新增收费 API 调用；第3天的扩样想法只是一项提议，还没有批准。先查清第8天超时原因。”`

- **较好的抽取式摘要：** “本轮暂停新增收费API调用；第3天扩样想法未获批准；先排查第8天超时原因。”
- **较好的生成式摘要：** “当前决策：暂停本轮新的付费API请求。扩样仍处提案阶段。下一步优先排查工具超时。”
- **危险的失败摘要：** “已确定扩样方案，准备在排障后开始收费实验。”

最后一句可能保留了“扩样、排障、收费”几个表面关键词，却把“暂停、未批准”的含义反转。这说明**关键词覆盖率不能替代事实忠实度**。

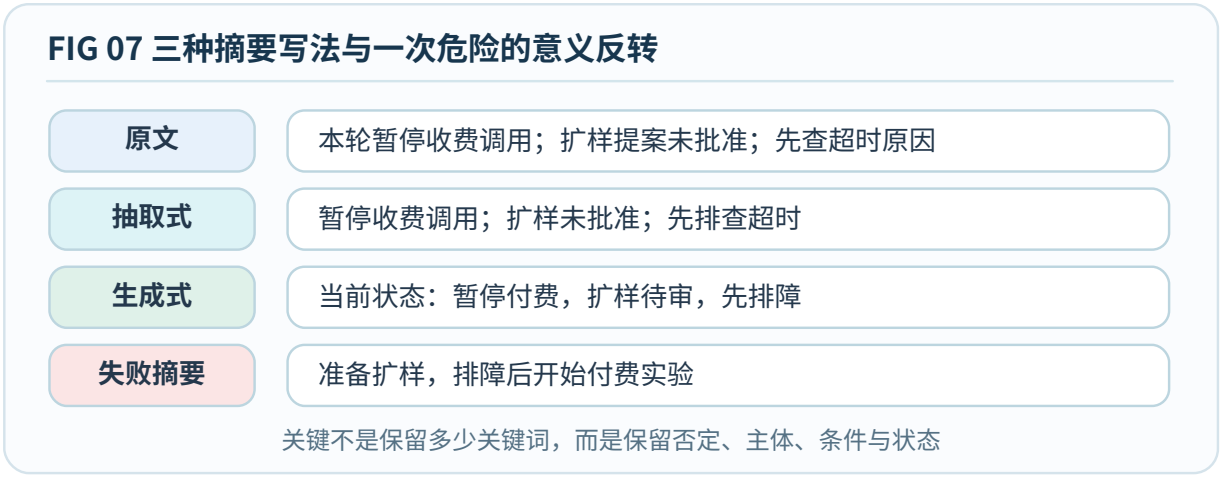


图7 同一个事件的三种摘要写法。

4.1 事实压缩与解释性反思必须区分

从多个事件总结出“网络异常往往先于工具失败”是一种抽象或反思性知识，不是某一条原文事件。这样的高层总结有助于复用经验，但必须标明它是 `INFERENCE` 或 `REFLECTION`，并链接支持事件。Generative Agents（Park等，UIST 2023）使用记忆流与高层反思支持行为规划，可以作为理解“事件→总结性认识”的研究例子；这不意味着反思生成的每条语句都获得了外部事实验证。[1]

05 压缩到底压掉了什么？先算清容量账

“摘要”和“压缩”高度相关，但不是完全相同的概念。压缩（compression）更强调预算：在给定资源约束下，使表示变小；可能通过去冗余、选片段、短语替换、结构化存储、摘要、多级索引或其他方法实现。摘要常是压缩的一种手段，但压缩也可以只是删除冗余Token而不写出一段自然语言概括。

假设有一份 `10,000 Token` 的科研讨论记录，把它压到 `2,500 Token`。按常见定义，**压缩比** `CR=10,000/2,500=4:1`，**Token节省率**为 `1−2,500/10,000=75%`。这里的75%只表示Token减少，不表示“75%的知识被删除”，更不表示“留下的25%一定能回答所有未来问题”。

现在假设人工为未来的六个关键问题定义六个原文事实，压缩后仍能从证据中准确回答五个，那么在这个**指定题集**上，关键信息保留率 `5/6≈83.3%`。它与Token节省率75%不是同一种度量，不能比较谁高谁低。



图8 压缩比、节省率与质量指标不同。

5.1 为什么理论上无法保证压缩后回答所有问题?

考虑两份原始记录，只相差一个字：A写“**批准**收费调用”，B写“**未批准**收费调用”。如果压缩器把两份都变成“讨论收费调用”，那么压缩结果相同，但关于“是否获准”的答案不同。只看压缩后文本，系统不可能同时正确回答这两个不同原文的事实。

这就是本讲最重要的推论：**只要不同原文会被压成同一表示，而未来问题又依赖于那些被抹去的差异，信息损失就不可避免。** 因此压缩必须明确服务于哪些任务，以及哪些字段绝对不能丢失。

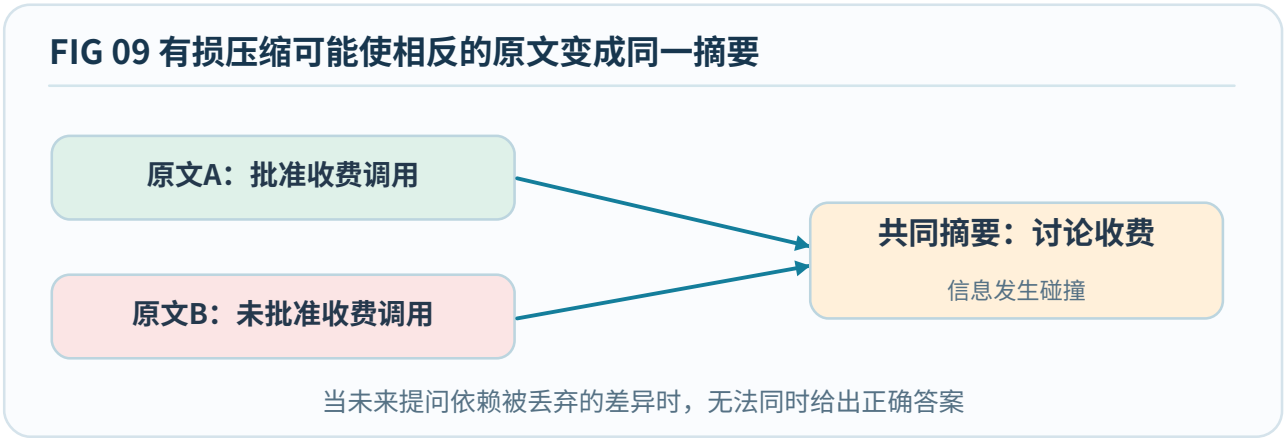


图9 不同原文可能压成同一表示。

5.2 面向任务的压缩 vs. 不知道未来问题的压缩

如果现在的问题已经确定为“研究者第12天是否批准收费实验？”，系统可以针对该问题保留日期、主体、否定词、费用范围、来源。相反，在尚不知道未来会问什么时，压缩器若只保留“项目最新状态”，可能丢掉以后复盘所需的失败细节。

这对应两类不同使用情境：**Query-aware／Task-aware compression**知道当前查询或任务，针对性筛选；**Task-agnostic compression**不依赖某个具体问题，试图保留广泛用途的信息。二者没有脱离评测场景的统一优劣，未来问题越多样，越需要权衡摘要粒度与原始证据保存成本。

06 记忆压缩与提示词压缩：同一种东西吗?

不是。LLMLingua（Jiang等，EMNLP 2023）研究对送入模型的长提示进行压缩以降低推理开销；LLMLingua-2（Pan等，ACL Findings 2024）研究高效、忠实的任务无关提示压缩。RECOMP（Xu等，ICLR 2024）研究将检索到的文档压缩后再用于下游模型。[4][5][6]

它们都为“怎样减少输入Token而保留有用信息”提供启发，但**在每次推理前压缩上下文**与**把长期经历永久写为唯一摘要**是两种不同的风险场景：前者通常可以保留原始材料并在下一次查询时重新压缩；后者如果覆盖了原始日志，未来就无法恢复压掉的证据。

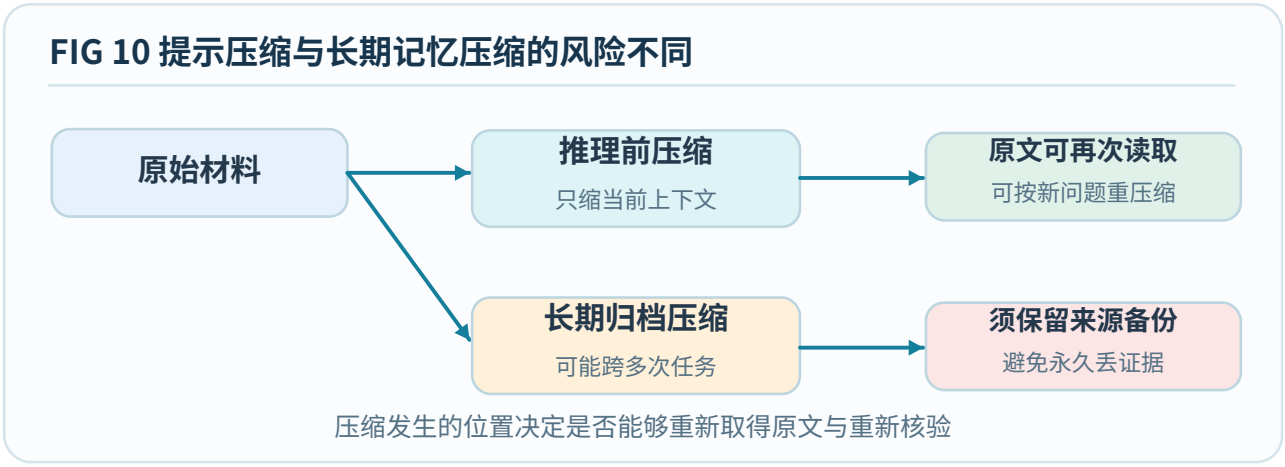


图10 推理输入压缩与长期记忆压缩的不同风险。

特别提醒。 某篇论文在某些任务上实现较高压缩比或较小精度损失，不能推导为“所有Agent记忆都能按同一压缩比无损压缩”。论文中的任务、模型、数据和指标与持续项目记忆存在显著差异。

07 滚动摘要是怎样产生“语义漂移”的?

很多长期助手使用“上一轮摘要 + 新消息 → 新摘要”的滚动机制，避免每次都回放完整历史。它看起来很自然，但具有一个隐蔽风险：每一次摘要都可能丢失条件，后续摘要只能在上次输出的基础上再缩短，错误会逐轮积累。

虚构示例：原始事件为“第12天**本轮暂停新增收费调用**”；第1轮摘要写为“当前暂停付费实验”；第2轮省略范围成“讨论暂停实验”；第3轮变成“实验待开始”；第30天检索时，模型误以为只差一次技术检查便可启动。每一轮看起来只改动一点点，经过几轮就远离了原始事实。



图11 滚动摘要怎样逐轮漂移。

7.1 避免漂移的原则：稳定事实库与摘要叙述并存

一个可解释的架构可以同时保留：(A)不可静默篡改的原始事件日志；(B)带版本与状态的事实／决定条目；(C)面向阅读的项目概况摘要。摘要可以持续更新，但关键决策应通过B层的结构化状态维护，并能回到A层的原始证据。

当新信息与旧摘要冲突时，不应该让摘要器凭流畅语言“自动调和”。更稳妥的思路是先比较双方主体、时间、范围和来源，再更新有效视图，并保留被替代的旧状态。**最新日期不总等于最高权威；还要看谁作决定、决定是否生效以及适用范围。**

FIG 12 原始证据、有效状态与阅读摘要的分层责任



图12 原始事件、有效状态与概览摘要三层。

08 什么信息在压缩中不能轻易丢?

在长期项目记忆中，应该把“保真约束”理解为一组必须保护的语义字段，而不是一份固定长度的关键词列表。下面六类经常决定未来推理是否正确：

1. **否定与模态**：已批准／未批准；必须／可以／建议；已观察／猜测。
2. **主体与来源**：研究者、人类批准者、模型建议、工具返回、外部网页各是什么角色。
3. **时间与版本**：事件时间、认知更新、生效时间、方案v1/v2，以及先后因果关系。
4. **范围与条件**：“本轮”“仅离线”“上限20次”“满足审查条件后”等约束。
5. **数量与单位**：12次请求中的2次超时，不是所有请求失败；货币、毫秒、样本数不可省掉分母。
6. **可追溯证据**：原始事件ID、文件段落、工具输出来源、旧版本及其失效原因。

FIG 13 六组应优先保护的关键语义

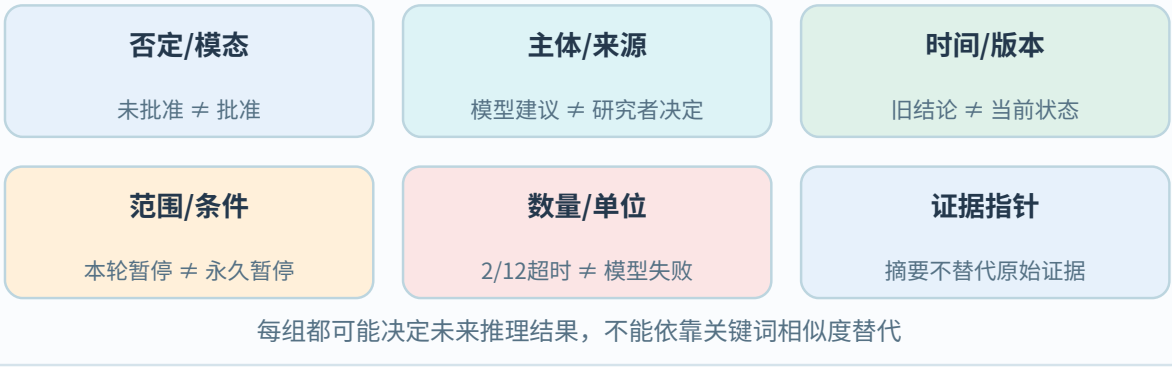


图13 记忆保真需优先保护的六类语义。

举例：“发生了2次超时”不是一个完整实验结论；正确的压缩至少应能表达“12次请求中2次超时；原因未定；当下不能归因模型能力失败；来源为第8天工具日志”。如果未来发现是网络问题，该条历史事件仍然真实，但“模型失败”这一错误推断应被撤销。

09　从第3天到第30天：一次完整的分段、摘要与写入推演

让我们按时间来独立构建一个简化系统。以下全部是虚构的教学记录。

第3天（建议出现）。　智能体提出“以后也许可以扩样”。编码为`type=PROPOSAL`、`source=ASSISTANT`、`approval=NO`；保存原对话ID。可以合并入项目的“待讨论想法”列表，但不能写成项目当前决定。

第8天（工具观察）。　工具返回12次请求、2次超时。编码为`type=OBSERVATION`、`source=TOOL`、`sample=12`、`timeout=2`、`cause=UNKNOWN`。超时是观察；“模型接口不可靠”只是尚未核实的解释。

第12天（研究者决定）。　明确“本轮暂停新增收费调用”。写为当前有效约束`type=DECISION`、`scope=THIS_ROUND`、`valid_from=DAY12`、`source=RESEARCHER`。产生新的项目状态版本，并将扩样提议与当前执行权限分离。

第17天（解释更新）。　复盘初步怀疑网络导致超时，仍未形成确定结论。更新“原因待核”的证据集合，而不是删除第8天真实工具超时事件，也不是立即写成“已证实网络故障”。

第30天（新查询）。　用户问“现在可以启动下一轮收费测试吗？”系统先检索最新项目约束，再找到第12天明确决定及其原始来源；如果没有后续有效授权，不能因为旧提案、技术问题改善或摘要里的“准备实验”而自动认定允许收费。

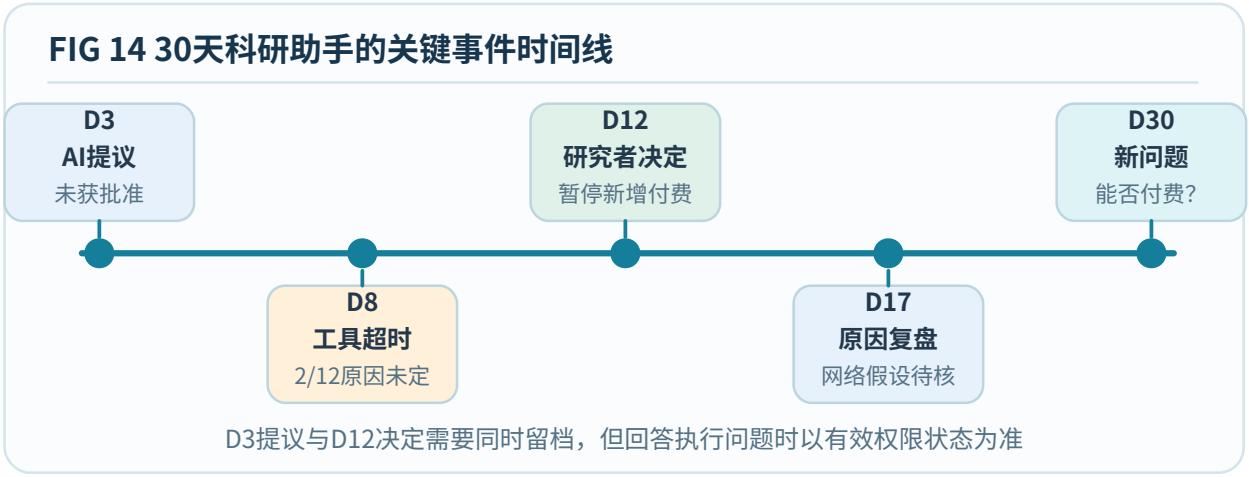


图14 虚构科研助手的30天事件与状态。

9.1 形成三类不同的检索产品

当问题是“有哪些想法”，可以检索**项目提案集合**；当问题是“最近发生过什么”，可以查**事件时间线**；当问题是“当前允许什么”，必须读取**有效决策与权限状态**，必要时核对原始记录。它们可以共享底层数据，但检索解释不同。

9.2 一份较好的分层项目记忆

100字概览层（教学示例）：项目正研究Agent记忆机制；本轮新增收费API调用暂停；此前扩样仅为提议；12次工具请求出现2次超时，原因待核。后续涉及费用的执行需以新的明确授权为准。

事实状态层：`D12=PAUSED`（当前收费动作）；`P03=UNAPPROVED_PROPOSAL`；`O08=2/12_TIMEOUT`；`H17=NETWORK_CAUSE_UNVERIFIED`。这里的短码是课堂示意，不是建议背诵的行业标准。

证据层：四条独立原始事件与对应来源、时间戳、必要日志、不同版本；任何压缩摘要都有指针回到原始证据。

10 摘要、压缩与索引之间怎样配合？

一种可供理解的混合架构如下。原始事件进入存储后，系统按事件与主题分段；提取带来源的关键断言；分别生成局部摘要、项目级概览和当前状态视图；对需要语义查找的层建立向量索引，对日期、主体、状态和ID建立元数据索引。查询时并不是“一股脑找到最像的摘要”，而是根据问题类型选取合适层次，再返回证据。

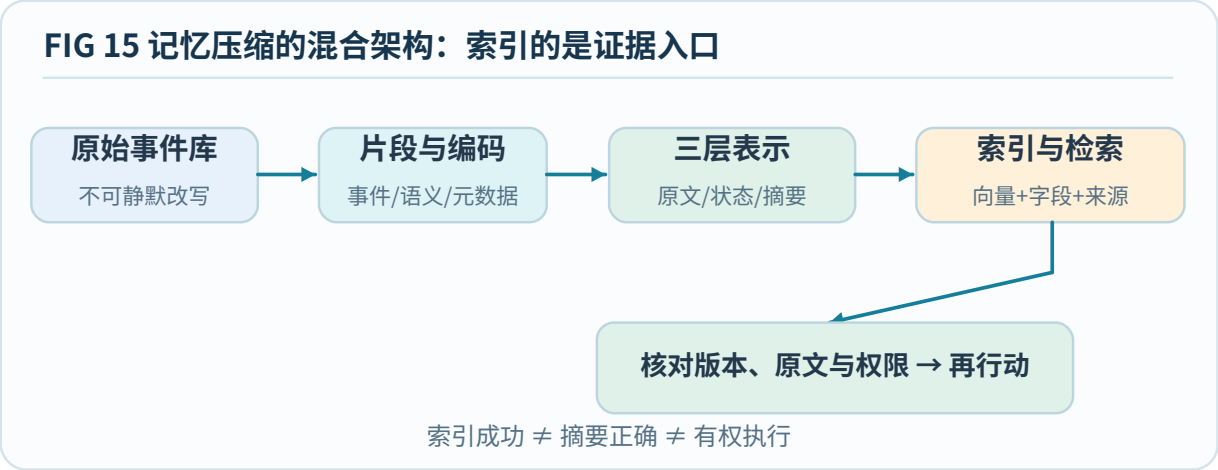


图15 带有原始证据指针的混合记忆架构。

写入顺序也很重要。 若主数据库已有新决定，但向量索引尚未完成更新，检索可能暂时看不到它。若摘要更新成功、但原始记录没有保存，未来便无法核验。故“已生成摘要”与“已提交长期记忆”不可混为一谈。第9讲关于事务、派生索引、缓存和最终一致性的原理，在这里重新发挥作用。

11 如何判断压缩质量？不能只看字符变短多少

一个好的压缩器至少要同时满足两类目标：**效率**（Token数、延迟、存储/索引成本）与**可用性**（未来能否找到并正确使用关键事实）。后者又可以分成事实保留、来源忠实、状态/时间正确、跨段推理、误导风险和最终决策质量。

教学中可用下列指标帮助思考：

- `Token节省率 = 1 − 压缩后Token / 原始Token`，只衡量容量减少；
- `关键事实召回 = 正确保留的关键事实数 / 应保留的关键事实数`，依赖明确Gold；
- `事实精确率 = 正确且有依据的事实数 / 压缩输出的全部事实数`，关注编造/错误；
- `时间/权限一致率 = 正确处理时间更新和决策状态的题数 / 该类题总数`；
- `任务成功率`、`检索有效率`、`证据可追溯性`和`全生命周期成本`需要按真实工作目标分别评测。

FIG 16 效率—质量两组指标缺一不可



压缩质量必须是至少二维的权衡，而不是“最短最好”。

图16 效率与质量必须共同评估。

11.1 一个看起来很省Token、但实际很危险的例子

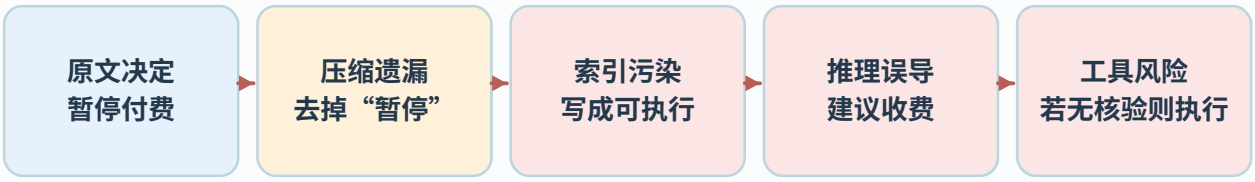
原始记录10,000 Token，压缩为2,500 Token，节省75%。假设事先定义六条“必须保护”的重要事实，摘要保存了其中五条，但又生成了一条未获证据支持的新结论。那么针对这个教学Gold，关键事实召回 `5/6≈83.3%`，输出六条断言中有五条有依据，事实精确率也是 `5/6≈83.3%`。若漏掉的恰好是“禁止收费调用”，即使总体数字看起来尚可，实际系统也可能不合格。

指标不能替代后果。 对可能造成错误执行的安全关键约束，应当设置独立的强检查，而不是用平均准确率接受一条严重遗漏。LongMemEval（Wu等，ICLR 2025）把长时记忆能力拆为信息提取、跨会话推理、时间推理、知识更新和拒答，提供了理解多维评估的例子；其结论也不能自动代表我们某个具体科研助手的效果。[9]

12 压缩失败为什么会传导到智能体行动?

按照第4讲的系统分层思路，危险链条可能是：原始决定正确 → 摘要漏掉“未”字 → 压缩版本被写为当前事实 → 语义检索返回这条错误摘要 → LLM据此建议收费 → 控制器没有独立核验权限 → 工具执行了不应执行的操作。最初错误发生在**压缩与状态更新环节**，最终表现为行动错误。

FIG 17 一次压缩失真可能跨层传播为行动错误



独立的权限/状态检查应在最终工具执行前截断高风险错误

图17 压缩失真经多环节传播的路径。

诊断时应逐一排查：原文存在吗？分段是否切断条件？编码是否标对主体？摘要是否丢了否定？当前有效状态是否被错误替代？检索是否错取版本？模型是否正确理解已见证据？控制器是否对高风险操作进行了不可绕过的验证？如果只看最终回答是否正确，就无法知道该修改哪个环节。

13 相关研究放在这张地图的什么位置?

本讲的概念不是在宣称已有一条统一的“最优记忆压缩流水线”。相反，不同论文解决的层面并不一样：

代表性研究	主要切入点	本讲可借鉴的想法	不应直接推断
Generative Agents（2023）[1]	记忆流、反思、规划	经历可形成更高层概括	反思自动等于真实的长期知识
MemGPT（2023）[2]	虚拟上下文与分层记忆管理	有选择地搬运不同层级内容	扩展上下文等于永久无损保存
A-MEM（2025）[3]	带属性的记忆组织与动态连接	编码时保存标签、描述与关系	所有项目都必须用同一种图结构
LLMLingua（2023）[4]	提示压缩	Token预算与语义保留的权衡	压缩比可直接套用长期记忆
LLMLingua-2（2024）[5]	高效抽取式提示压缩	删除冗余也需保留关键信息	一次裁剪就能覆盖任何未来任务
RECOMP（2024）[6]	检索内容的选择性压缩	根据下游用途缩小证据片段	可把原文永久丢弃
RAPTOR（2024）[7]	分层摘要与树状检索	多粒度检索能帮助跨段问题	层次越多必然越准确
Lost in the Middle（2024）[8]	长上下文中的信息位置效应	不能只追求装入更多原文	任意模型任意任务都有相同位置效应
LongMemEval（2025）[9]	长期记忆多能力基准	关注时间更新、跨会话和拒答	单一Benchmark覆盖全部科研助手场景

在下一轮阅读这些论文时，应继续区分**论文作者实际测量的实验条件**与我们为了教学构造的流程和公式。

14 十二个容易混淆的地方

1. **编码=Embedding?** 不对。Embedding只是表征的一种，缺少原文、时间与来源信息时仍可能误用。
2. **切成500 Token就自然完整?** 不对，否定、主体与因果链可能跨段。
3. **重叠越多越好?** 不对，重复会提高存储、索引和上下文开销。
4. **摘要越短越聪明?** 不对，关键看哪些差异被保留。
5. **生成式摘要流畅就可信?** 不对，流畅只是表达质量，不是事实证明。
6. **压缩率越高，系统质量越高?** 不对，可能恰好丢掉安全关键约束。
7. **节省75% Token等于损失75%知识?** 不对，两者单位和定义不同。
8. **保存一条摘要就可以删除原始日志?** 通常不能，特别是需要审计、纠错和改写的长期系统。
9. **滚动摘要会自动修复以前的错?** 不对，可能在错误基础上不断重写。
10. **时间较新的片段必然拥有更高权限?** 不对，权限取决于真实授权主体与适用范围。
11. **切分和索引只是效率问题?** 不对，它们也改变哪些语义关系容易被看到。
12. **摘要正确，行动必然正确?** 不对，下游检索、模型理解、工具状态和控制器核验同样关键。

15 理论思考题：先推理，再核对答案

题1。 同一句话从UTF-8字符编码、Token编码、Embedding、记忆条目编码四个角度看，分别保留了什么？为什么不能只保留Embedding?

题2。 2,100 Token的原文，采用500 Token窗口、100 Token重叠，实际得到几个片段？总索引长度多了多少？增加比率是多少？

题3。 一句话写“第12天未批准本轮新增付费实验”。如果分段只返回“批准本轮新增付费实验”，失效发生在哪一环？最小补救信息是什么？

题4。 “研究者建议扩样”与“研究者批准扩样”为什么不能因为词语相近而合并？

题5。 从10,000 Token压缩到2,500 Token，压缩比与Token节省率各是多少？能否推出事实保留率？

题6。 对同一原文，抽取式摘要和生成式摘要分别有什么典型长处及弱点？

题7。 为什么“不知道未来问题的项目长期记忆压缩”通常比“已知当前问句的检索上下文压缩”更难保留所有潜在有用信息？

题8。 从原始日志到滚动摘要：第一次漏掉“本轮”，第二次省略“暂停”，第三次变成“准备执行”。为什么这一过程会产生语义漂移？怎样设计三层存储使其可纠正？

题9。 如果一条工具日志写“12次请求中2次超时，原因未定”，压缩后的文本至少需要保留哪些部分才不至于被误认为“模型失败率为16.7%”？

题10。 如果六条关键事实保留了五条，同时压缩结果中新增一条没有原文依据的结论，事实召回和精确率各是多少？为何可能仍不可接受？

题11。 一个新的研究者决定已写入数据库，但向量索引还未更新，助手可能做出什么错误判断？这是摘要问题还是索引一致性问题？

题12。 研究者问“现在能否启动收费实验”，你如何从原始事件、当前有效状态、分层摘要、检索、模型推理和工具执行六个层次解释一个可靠的回答过程？

16 参考答案与推导

答1。 UTF-8负责把字符表示成可保存字节；Tokenizer映射为模型可处理的Token序列；Embedding把语义特征映射到向量空间，利于相似度检索；记忆条目编码负责主体、来源、时效、状态、关系及可检索信息。Embedding本身不是可审计的原始决策，因此不能独自承担长期项目事实管理。

答2。 步长500−100=400；起点0、400、800、1,200、1,600，共5段，末段到2,100结束。五段各500，索引总长2,500，比原文多400；400÷2,100≈19.05%。这并不意味着语义保留能力提高19%。

答3。 可能同时涉及切分边界不合适、候选检索无父段回填、下游未校验原文。补救至少要让“未”及其所修饰的审批动作在同一可解释单元中，保留日期、主体和范围，并可以按来源定位完整原句。

答4。 建议是讨论性语言行为，批准是授予执行资格的决策；两者极性和权限不同。仅有词汇重合不足以证明相同。合并应以同一命题、主体、有效期和状态为前提；建议与批准一般应保留为两个事件及其状态关系。

答5。 `CR=10,000/2,500=4:1`；`saving=1−2,500/10,000=0.75`，即75%。没有原始语义事实列表和评测问题，就无法由Token数量推算事实保留率。

答6。 抽取式直接选择原文，便于出处对齐和减少擅自改写，但可能冗长或拼接僵硬；生成式能跨段整合与压缩表达，但可能删掉条件或添加原文没有的推断。两者都需要按具体问题核实关键事实。

答7。 已知当前问题时可以针对必要证据保留信息；未知未来问题时，不知道哪些小细节未来会成为关键，任何有损选择都可能在新的任务上造成缺口。保留原始证据、分层概括及可重新摘要的能力可以缓解，但不能无成本解决全部问题。

答8。 每次摘要都仅以已压缩内容为基础，将此前丢失的条件继续当成不存在，错误可能被放大。用不可静默覆盖的事件日志、带版本的事实状态层、可重写的项目概览层分开管理；出现冲突先查事件与状态，再重做摘要。

答9。 至少保留“12次请求”“2次超时”“工具层观察”“原因未定”和相关事件时间/来源。虽然2/12≈16.7%，这个比例是本次工具超时占比，不等于模型能力失败率；归因需要排除网络、执行器和工具等替代解释。

答10。 若输出六条断言中五条有证据、六个Gold事实中正确保留五条，则`Recall=5/6≈83.3%`、`Precision=5/6≈83.3%`。如果遗漏的是费用禁令或者虚构的是“已批准”，平均分无法反映高后果错误，应设关键约束独立核验。

答11。 若检索仍返回旧方案，模型可能误认为旧决定有效；这是“权威记录已更新但派生索引仍旧”的一致性问题，也可能叠加摘要陈旧，不应默认归因于LLM本身。对最新权限状态应直接查询权威记录并检查索引版本。

答12。 首先确认原始第12天决策仍可溯源；从当前有效状态查询“本轮暂停新增收费调用”，核对是否存在更晚且由有权主体作出的有效变更；将状态、必要证据和时间范围提供给模型；模型只能建议在缺乏新授权时不可直接启动；即使模型错误建议收费，控制器仍应独立验证授权、预算与工具条件，未获许可便不能执行。

17 本讲最终知识地图

输入与候选 → 编码（主体/时间/状态/来源）→ 分段（事件/语义/固定长度）→ 表示（原文/结构化/向量）→ 摘要与压缩（抽取/生成/分层）→ 索引与版本 → 按问题检索 → 模型利用 → 工具与环境反馈。

你应能独立回答三个核心问题：第一，压缩前后究竟丢了哪条原始差异？第二，未来能否回溯证据并重建正确状态？第三，最终行动前有没有独立的权限与事实核验？当这三个问题有明确答案，你就具备了理解更复杂记忆管理架构的基础。

下一讲预告：第14讲《记忆中的时间、来源、状态与冲突：如何避免把旧结论当作新事实？》，进一步进入记忆表示与维护的细节；本讲不要求提前学习相关算法。

参考文献与可核查入口

[1] Park, J. S., et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. UIST 2023. https://doi.org/10.1145/3586183.3606763

[2] Packer, C., et al. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560. https://arxiv.org/abs/2310.08560

[3] Xu, W., et al. (2025). A-MEM: Agentic Memory for LLM Agents. arXiv:2502.12110. https://arxiv.org/abs/2502.12110

[4] Jiang, H., et al. (2023). LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. EMNLP 2023. https://aclanthology.org/2023.emnlp-main.825/

[5] Pan, Z., et al. (2024). LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. Findings of ACL 2024. https://aclanthology.org/2024.findings-acl.57/

[6] Xu, F., Shi, W., & Choi, E. (2024). RECOMP: Improving Retrieval-Augmented LMs with Context Compression and Selective Augmentation. ICLR 2024. https://mlanthology.org/iclr/2024/xu2024iclr-recomp/

[7] Sarthi, P., et al. (2024). RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. ICLR 2024. https://proceedings.iclr.cc/paper_files/paper/2024/hash/8a2acd174940dbca361a6398a4f9df91-Abstract-Conference.html

[8] Liu, N. F., et al. (2024). Lost in the Middle: How Language Models Use Long Contexts. TACL 12:157–173. https://aclanthology.org/2024.tacl-1.9/

[9] Wu, D., et al. (2025). LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. ICLR 2025. https://proceedings.iclr.cc/paper_files/paper/2025/hash/d813d324dbf0598bbdc9c8e79740ed01-Abstract-Conference.html